itthynk

SOFTWARE DEVELOPMENT INTERNSHIP PROGRAMME · COHORT 2026

Demo moved · now Monday 14 Sep, not Friday

Event Management System — Week 4 Feedback

This is the most complete submission of the five, and it shows — hashing, sessions, cookies, and the 401/403 split are all handled with real care, in some places more carefully than the brief actually asked for. There's one lesson worth sitting with properly, because it's the exact kind of mistake that's easy to make even when everything else is done well.

Repo · `cohort-2026/event-management-system`

Sprint work · Authentication & Authorization, Mon 7 – Fri 11 Sep 2026

Demo · Monday 14 Sep 2026 (moved from Friday)

WHAT THIS DOCUMENT IS — AND ISN'T

This is **feedback, not a fix-it list.** Every item below is here to help you see where your own reasoning went wrong and think through a better way to approach it — not to hand you code to paste in. Each one points at the real file and line in your repo so you can go look at it yourselves, and some show the actual code that's there right now — not because copying a replacement is the point, but because seeing your own code named directly is more useful than a description of it. The short examples further down are illustrative, to make an idea concrete, not to be copied.

What to demo on Monday

The brief's own sequence, walked through against what this repo actually does today — from the same audit this feedback is drawn from.

Step 1 Register a new account

Ready

ON SCREEN

Clean form, validates email format and password length live.

IN THE CODE / TERMINAL

201, bcrypt hash confirmed, response never includes `passwordHash`.

Step 2 Log in

Ready

ON SCREEN

Redirected to a role-appropriate dashboard.

IN THE CODE / TERMINAL

`Set-Cookie` with httpOnly, `sameSite=lax`, and `secure` only when `NODE_ENV=production` — correctly conditional, worth pointing out.

Step 3 Refresh the page

Ready

ON SCREEN

Stays signed in.

IN THE CODE / TERMINAL

`GET /api/auth/me` runs once on mount to rehydrate the session.

Step 4 Visit a protected page as a guest

Ready

ON SCREEN

Redirected via `GuestRoute`/`ProtectedRoute`.

IN THE CODE / TERMINAL

—

Step 5 Log in as an ordinary attendee, try an admin-only route

Ready

ON SCREEN

Blocked, with a clear message.

IN THE CODE / TERMINAL

Network tab shows 403, not 401 — the correct distinction, and a good moment to show off `requireAuth` vs `requireRole`.

Step 6 Log out

Ready

ON SCREEN

Returns to a signed-out state.

IN THE CODE / TERMINAL

Cookie cleared server-side via `res.clearCookie`.

Step 7 “Break it on purpose”: register with role set to ADMIN directly, bypassing the form

Currently exploitable

ON SCREEN

N/A — this is a terminal/Postman-only step.

IN THE CODE / TERMINAL

`curl -X POST /api/auth/register -d '{"role":"ADMIN", ...}'` currently succeeds

and returns 201 with the admin role granted.

Fix this before Monday (Solution 1) — or if you're deliberately demoing it as your "what we tried on Thursday" answer, be ready to show the fix immediately after.

RECOMMENDED ORDER TO WORK THROUGH THIS

WHEN	ISSUE	WHERE
1st	Restrict the role field on registration	Lesson 1
2nd	Fail fast on a missing JWT_SECRET	Lesson 2
next sprint	Wire requireOwnership into the first real route that needs it	Lesson 3

Only the first fix is demo-critical — the role escalation is a real security bug, the other two are hardening for what comes next.

Where this went wrong, and how to think about it differently

In the order it'll matter most — not necessarily the order you'll want to tackle it.

1 The form was trusted to say who someone is allowed to be

`backend/src/controllers/authController.ts:32,58`

WHAT WENT WRONG

> The registration screen only offers two role choices in its dropdown — but the server accepts whatever role is sent in the request, not just the two the screen shows. Anyone who skips the form and sends the request directly can currently ask to be registered as an administrator, and the server will agree.

WHAT'S ACTUALLY THERE RIGHT NOW

The audit's finding at these two lines: `role` is read straight from `req.body` on registration with no allowlist check, so `{"role":"ADMIN"}` sent directly to the endpoint is accepted as-is — even though the frontend's own form only ever offers Attendee or Vendor.

WHY IT MATTERS

> *It's like a hotel check-in form that only lists "Standard Room" and "Deluxe Room" as options — but the front desk will happily hand over a penthouse key to anyone who simply writes "Penthouse" in the box by hand, because nobody at the desk actually checks the request against what's really available.*

A BETTER WAY TO THINK ABOUT IT

> A form limiting the *choices someone sees* is a UI convenience — it guides an honest user. It is never the thing that actually decides what's allowed; that decision has to be made again on the server, independent of whatever the client sent, because the client can always be bypassed entirely. Anywhere a permission or role is being set, ask: "if someone skipped the form and sent this by hand, would the server still refuse it?"

EXAMPLE

illustrative only

```
// instead of trusting whatever role arrives:
const { role } = req.body;

// decide it server-side, regardless of what was sent:
const ALLOWED_SELF_SIGNUP = ["ATTENDEE", "VENDOR"];
const role = ALLOWED_SELF_SIGNUP.includes(req.body.role) ? req.body.role :
"ATTENDEE";
```

2 A fallback value that quietly became the real one

`backend/src/config/index.ts:8`

WHAT WENT WRONG

> If the signing secret for sessions isn't set in the environment, the code quietly uses a fixed default value instead of refusing to run. It's a small thing, and it's already committed to the repository — which means the "fallback" is really just a publicly known password waiting to be relevant.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
jwtSecret: process.env.JWT_SECRET ?? "change_this_secret",
```

WHY IT MATTERS

> *It's a spare key kept under the doormat specifically so the house is never accidentally left unenterable — except the doormat is visible from the street, and anyone can look under it. The intention (never get locked out) is reasonable. The execution quietly removes the lock.*

A BETTER WAY TO THINK ABOUT IT

> For anything that's meant to be secret, prefer a loud failure over a quiet fallback — have the program refuse to start rather than substitute something that happens to work today. A crash on startup gets noticed immediately, in a way a working-but-insecure default never does.

EXAMPLE

illustrative only

```
jwtSecret: process.env.JWT_SECRET ?? (() => {
  throw new Error("JWT_SECRET is not set");
})(),
```

3 A safety check was built, and then never actually connected to anything

`backend/src/middleware/auth.ts:58-80 (requireOwnership) · backend/src/routes/ticketRoutes.ts:7-10`

WHAT WENT WRONG

> There's a well-built piece of middleware (requireOwnership) that checks whether someone is actually allowed to access a specific record, not just any record — but it isn't used anywhere yet. One of the route files even has a comment planning a by-id lookup for 'any authenticated user', without mentioning ownership at all.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, the route comment is quoted; requireOwnership is real but unused

```
// backend/src/routes/ticketRoutes.ts:7-10
// GET /api/tickets/:id — View a single ticket (any authenticated user)

// backend/src/middleware/auth.ts:58-80 — exists, works, used nowhere:
export function requireOwnership(getResourceOwnerId) { ... }
```

WHY IT MATTERS

> *It's like fitting a peephole to a door and never actually looking through it before opening — the tool to check who's there exists, it's just not part of the routine yet.*

A BETTER WAY TO THINK ABOUT IT

> Since the mechanism already exists and works, the cheapest moment to use it is right when the first route that needs it gets built — wiring it in from the start, rather than adding it as an afterthought once a route without it is already working and harder to revisit.

Checklist — work through this before Monday

One box per lesson above, phrased as something you can actually check off. Tick it once you can point to where it's true in your own code — not once you've read the explanation. Items marked **Critical** are where the audit found a real gap; be ready to show it live, not describe it.

- [ ] **Does the server reject (or ignore) a self-registration request that asks for role: ADMIN?** CRITICAL
- [ ] **Does the app refuse to start if JWT_SECRET is missing, instead of using the committed default?** CRITICAL
- [ ] Does requireOwnership get used on at least one route where 'any signed-in user' shouldn't mean 'any record'?